

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ESCOM

Des. de Aplicaciones Web:
Client and Backend Development Frameworks

Reporte de práctica

"Instalación del Java Development Kit"

Alumno

José Bryan Omar Jiménez Velázquez

Grupo

7CM2

Profesor

Dr. José Asunción Enríquez Zárate



Ciudad de México, agosto de 2026

josebryanomar2004@gmail.com

Índice

1. Introducción **1**

1.1. Contexto y propósito 1
1.2. Por qué hace falta configurar `JAVA_HOME` 1
1.3. Objetivos 1
1.4. Entorno de trabajo 1
1.5. Nota sobre la distribución elegida 2

2. Desarrollo **2**

2.1. Fase 1: verificación del estado inicial 2
2.2. Fase 2: desinstalación de versiones previas 3
2.3. Fase 3: descarga e instalación del JDK 5
2.3.1. Actualización de los índices de paquetes 5
2.3.2. Consulta de la versión disponible 6
2.3.3. Instalación 6
2.3.4. Verificación de la instalación 7
2.4. Fase 4: configuración de las variables de entorno 7
2.4.1. Identificación de la ruta de instalación 7
2.4.2. Definición de las variables 8
2.4.3. Aplicación y verificación 9
2.4.4. Gestión de varias versiones instaladas 9
2.5. Fase 5: verificación funcional 10

3. Conclusiones **11**

3.1. Sobre el proceso 11
3.2. Errores posibles durante el proceso y su solución 12
3.3. Recomendaciones 12
3.4. Sobre la distribución utilizada 13

Referencias **13**

1. Introducción

1.1. Contexto y propósito

Java no se instala como un programa único. Lo que se instala es una plataforma formada por tres piezas que conviene distinguir desde el principio, porque confundirlas es el origen de la mayoría de los problemas durante el proceso:

- La **JVM** (*Java Virtual Machine*) es la máquina virtual que ejecuta el *bytecode*. Es lo que hace que un programa compilado en una computadora funcione en otra con un sistema operativo distinto.
- El **JRE** (*Java Runtime Environment*) agrupa la JVM más las bibliotecas estándar. Con él se pueden **ejecutar** aplicaciones Java, pero no escribirlas.
- El **JDK** (*Java Development Kit*) contiene todo lo anterior más las herramientas de desarrollo: el compilador `javac`, el depurador `jdb`, el empaquetador `jar`, el generador de documentación `javadoc` y, desde Java 9, el ensamblador de imágenes `jlink`.

La distinción importa porque muchas distribuciones de Linux traen preinstalado únicamente el JRE. Un equipo así puede abrir cualquier aplicación Java y, sin embargo, al escribir `javac Programa.java` responde `command not found`. Ese es exactamente el estado inicial de la máquina documentada en este reporte.

1.2. Por qué hace falta configurar `JAVA_HOME`

Instalar el JDK deja disponibles los comandos `java` y `javac` en la terminal, y para un ejercicio de clase eso podría bastar. El problema aparece en cuanto se usa cualquier herramienta del ecosistema.

Maven, Gradle, Apache Tomcat, Apache Ant, Elasticsearch, Jenkins y los entornos de desarrollo como IntelliJ IDEA o Eclipse **no buscan el compilador en el** `PATH`: consultan una variable de entorno llamada `JAVA_HOME` que debe apuntar al **directorio raíz** de la instalación del JDK. Si esa variable no existe o apunta al lugar equivocado, esas herramientas fallan con mensajes del estilo `JAVA_HOME is not set and no 'java' command could be found`, aunque `java -version` funcione perfectamente en esa misma terminal.

Hay una segunda razón, menos evidente pero igual de importante: una computadora puede tener **varias versiones del JDK instaladas al mismo tiempo**. Es lo normal en un equipo de desarrollo, donde un proyecto heredado exige Java 8 y otro requiere Java 21. En ese escenario, `JAVA_HOME` es el mecanismo que declara cuál de todas es la que debe usarse, y el sistema de alternativas de Debian y Ubuntu (`update-alternatives`) es el que resuelve qué binario responde al escribir `java`.

1.3. Objetivos

De acuerdo con lo solicitado en la práctica:

1. **Identificar** las variables de entorno necesarias para la correcta ejecución del Java Development Kit, en particular `JAVA_HOME` y su relación con `PATH`.
2. **Configurar** dichas variables de forma persistente en un sistema GNU/Linux, verificando que la configuración sobreviva al cierre de la terminal y al reinicio del equipo.
3. **Documentar** el procedimiento completo —verificación del estado inicial, desinstalación de versiones previas, instalación, configuración y comprobación— con evidencia gráfica de cada etapa.

1.4. Entorno de trabajo

Tabla 1.1: Características del equipo utilizado

Elemento	Valor
Sistema operativo	Ubuntu 22.04.5 LTS (Jammy Jellyfish)
Arquitectura	x86_64 (64 bits)
Intérprete de comandos	Bash
Estado inicial de Java	OpenJDK 11 JRE (solo entorno de ejecución, sin compilador)
Versión a instalar	OpenJDK 21 LTS
Método de instalación	Gestor de paquetes APT, repositorio *jammy-updates*
Privilegios requeridos	Cuenta con acceso a `sudo`

1.5. Nota sobre la distribución elegida

La práctica menciona *Oracle Java Development Kit*. En este reporte se documenta la instalación de **OpenJDK 21**, y conviene justificar la decisión porque no es una omisión sino una elección técnica.

Desde Java 7, **OpenJDK es la implementación de referencia del lenguaje**: el código fuente del que parte Oracle para construir su propia distribución es el mismo proyecto OpenJDK. Ambas superan el conjunto de pruebas de compatibilidad TCK, de modo que un programa compilado con una funciona idénticamente en la otra. Las diferencias relevantes son de licencia y de soporte comercial, no de comportamiento:

- **OpenJDK** se distribuye bajo licencia GPLv2 con excepción de enlazado, es de uso libre en cualquier contexto y llega empaquetado en los repositorios oficiales de Ubuntu, lo que simplifica la instalación y las actualizaciones de seguridad.
- **Oracle JDK** se distribuye bajo la *Oracle No-Fee Terms and Conditions*, que permite el uso gratuito con restricciones, y requiere descargar el paquete manualmente aceptando la licencia.

Todo lo relativo a `JAVA_HOME`, al `PATH` y al sistema de alternativas —que es el objetivo de aprendizaje de la práctica— es **idéntico** en ambos casos; lo único que cambia es la ruta de instalación. En la sección de conclusiones se retoma esta comparación.

2. Desarrollo

El procedimiento se organizó en cinco fases. Cada una incluye los comandos ejecutados y la evidencia gráfica del resultado obtenido.

2.1. Fase 1: verificación del estado inicial

Antes de instalar cualquier cosa conviene averiguar qué hay. Saltarse este paso es la causa más común de terminar con dos o tres versiones del JDK conviviendo sin que el usuario sepa cuál está usando realmente.

```
# Version del entorno de ejecucion
java -version

# Version del compilador: si responde "command not found",
# lo que hay instalado es el JRE y no el JDK
javac -version

# Contenido de la variable de entorno: si sale vacio, no esta configurada
echo "JAVA_HOME = [$JAVA_HOME]"

# Ubicacion real del binario, siguiendo los enlaces simbolicos
which java
readlink -f $(which java)
```

Código 2.1: Diagnóstico del estado inicial

En el equipo utilizado, `java -version` respondió con OpenJDK 11, mientras que `javac -version`

devolvió `command not found` y `JAVA_HOME` apareció vacía. Es decir: había un entorno de ejecución, pero no un kit de desarrollo.



Figura 2.1: Estado inicial del sistema: existe la JVM pero no el compilador, y `JAVA_HOME` está vacía

También conviene inventariar los paquetes relacionados que el gestor tiene registrados, porque son los que habrá que retirar:

```
# Paquetes de Java registrados en el sistema
dpkg -l | grep -iE "jdk|jre"

# Versiones presentes en el directorio estandar de JVM
ls -la /usr/lib/jvm/
```

Código 2.2: Inventario de paquetes Java instalados

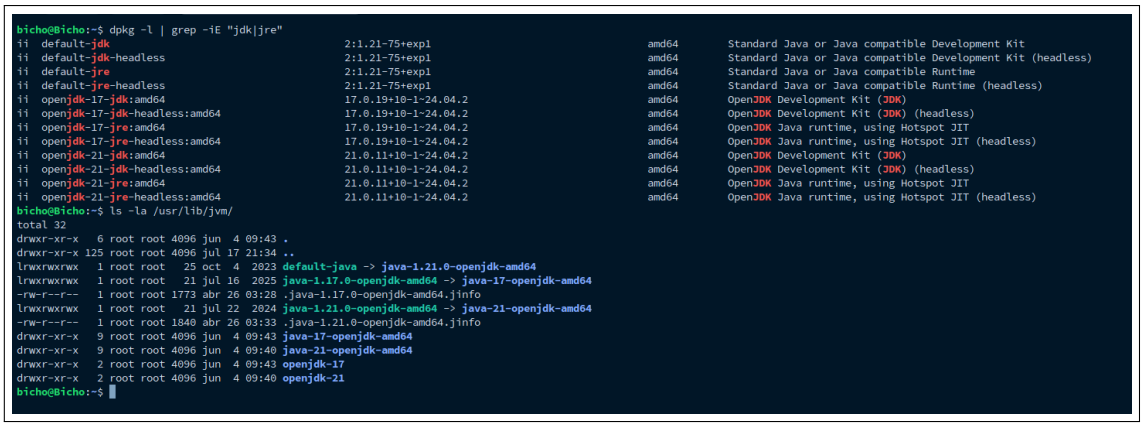


Figura 2.2: Paquetes relacionados con Java presentes antes de la instalación

2.2. Fase 2: desinstalación de versiones previas

Esta fase es opcional en un equipo limpio, pero es la que evita el escenario más confuso de todos: tener varias versiones instaladas y no saber cuál responde.

```
# --purge elimina tambien los archivos de configuracion, no solo el paquete
sudo apt remove --purge "openjdk-11-*" -y

# Retira las dependencias que quedaron huerfanas
sudo apt autoremove -y

# Comprobacion: ambos comandos deben responder "command not found"
java -version
javac -version
```

Código 2.3: Retiro de la versión anterior

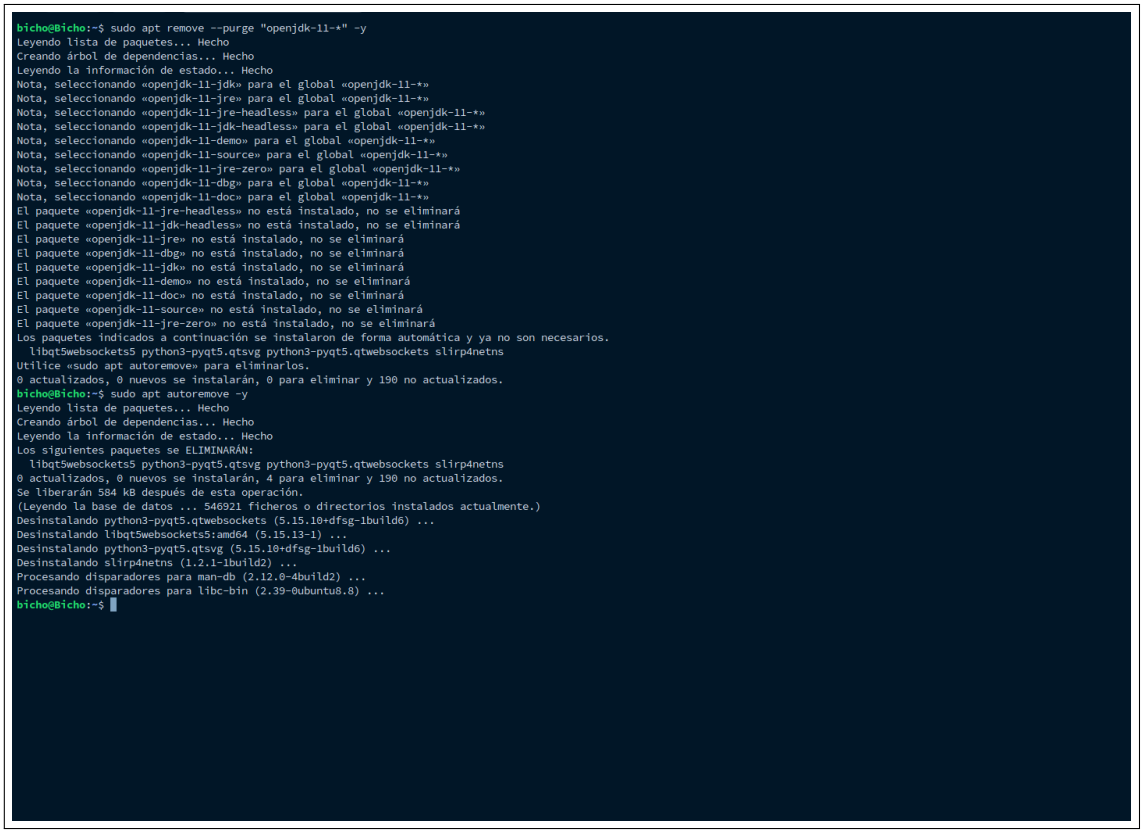


Figura 2.3: Salida del proceso de desinstalación de OpenJDK 11

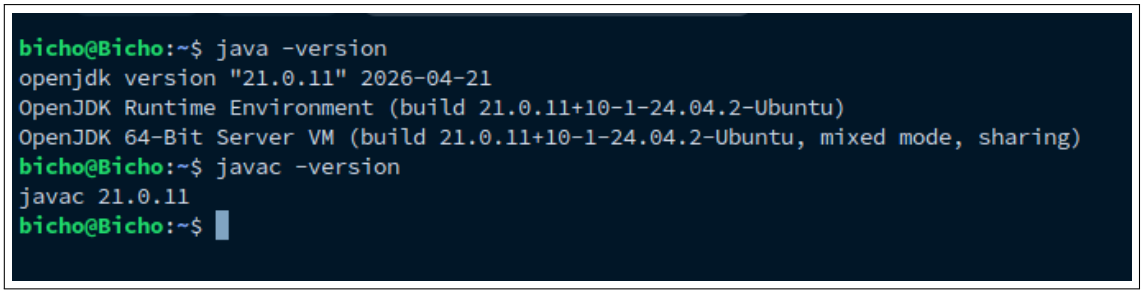


Figura 2.4: Verificación de que el sistema quedó sin ninguna versión de Java

Incidencia registrada durante esta práctica. Al ejecutar el primer comando, el sistema respondió:

```
$ sudo apt remove --purge "openjdk-11-*" -y
E: se interrumpio la ejecucion de dpkg, debe ejecutar manualmente
   «sudo dpkg --configure -a» para corregir el problema
```

Código 2.4: Error por una operación de dpkg interrumpida

El mensaje no tiene relación con Java: indica que una instalación anterior de **cualquier** paquete quedó a medio configurar, y APT se niega a realizar operaciones nuevas mientras la base de datos de paquetes esté en ese estado inconsistente. La reparación es la que el propio mensaje sugiere:

```
# Termina la configuracion pendiente
sudo dpkg --configure -a

# Repara dependencias rotas, si las hubiera
sudo apt --fix-broken install

# Comprueba que APT vuelve a operar con normalidad
sudo apt update
```

Código 2.5: Reparación del estado de dpkg

Si el problema persiste, `sudo dpkg -audit` enumera los paquetes que quedaron en estado inconsistente, lo que permite identificar cuál es el que falla.

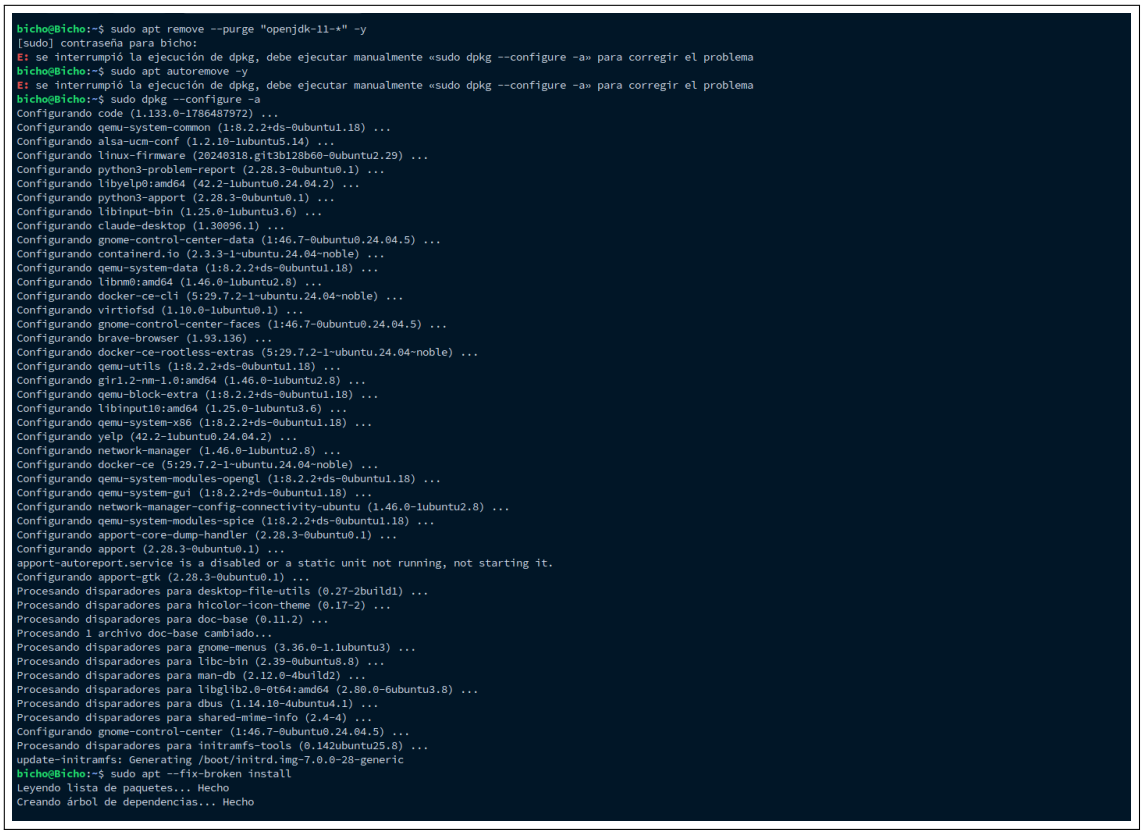


Figura 2.5: Error por una operación de `dpkg` interrumpida y su reparación

Advertencia importante. En Ubuntu, varias aplicaciones de escritorio dependen de Java. Si `apt autoremove` propone eliminar programas que se están usando, conviene revisar la lista antes de aceptar. Cuando existan dudas, la alternativa prudente es **no desinstalar**: instalar la versión nueva junto a la anterior y seleccionar cuál usar con `update-alternatives`, como se explica en la fase 4.

2.3. Fase 3: descarga e instalación del JDK

2.3.1. Actualización de los índices de paquetes

APT trabaja con una copia local del catálogo de los repositorios. Si esa copia está desactualizada, el sistema puede no encontrar el paquete o intentar descargar una versión que ya no existe en el servidor —un error frecuente que se manifiesta como `404 Not Found` a mitad de la instalación—.

```
sudo apt update
```

Código 2.6: Actualización del catálogo de paquetes



Figura 2.6: Actualización de los índices de los repositorios

2.3.2. Consulta de la versión disponible

Antes de instalar conviene confirmar qué versión exacta ofrece el repositorio y de cuál procede:

```
# Version candidata y repositorio de origen
apt-cache policy openjdk-21-jdk

# Todas las versiones del JDK disponibles en los repositorios
apt search --names-only "^openjdk-.*-jdk$"
```

Código 2.7: Consulta de la versión candidata

En Ubuntu 22.04 el paquete proviene del repositorio *jammy-updates*, en la sección *universe*, con la versión `21.0.11+10-1~22.04.2`.

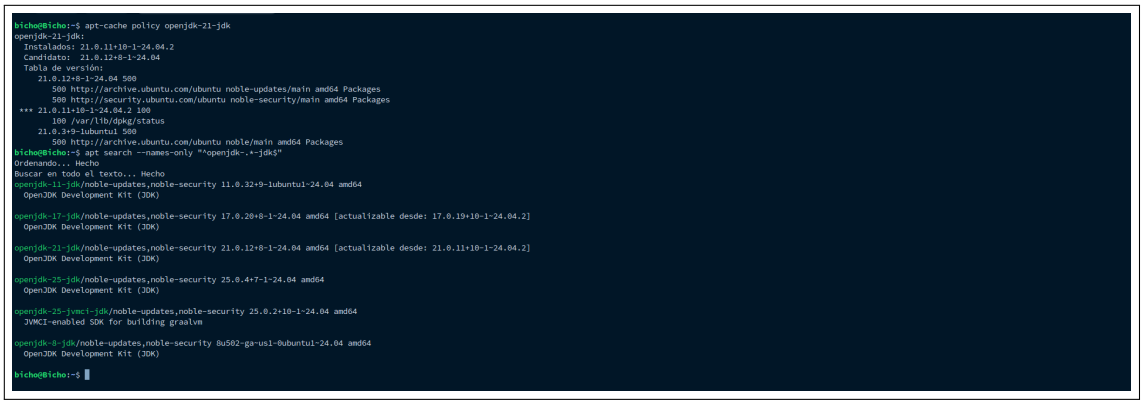


Figura 2.7: Versión candidata de `openjdk-21-jdk` y repositorio del que proviene

2.3.3. Instalación

```
sudo apt install openjdk-21-jdk -y
```

Código 2.8: Instalación del JDK

Nótese que el paquete es `openjdk-21-jdk` y no `openjdk-21-jre`. Instalar el segundo por descuido deja el sistema en el mismo estado del que se partió: con máquina virtual pero sin compilador.

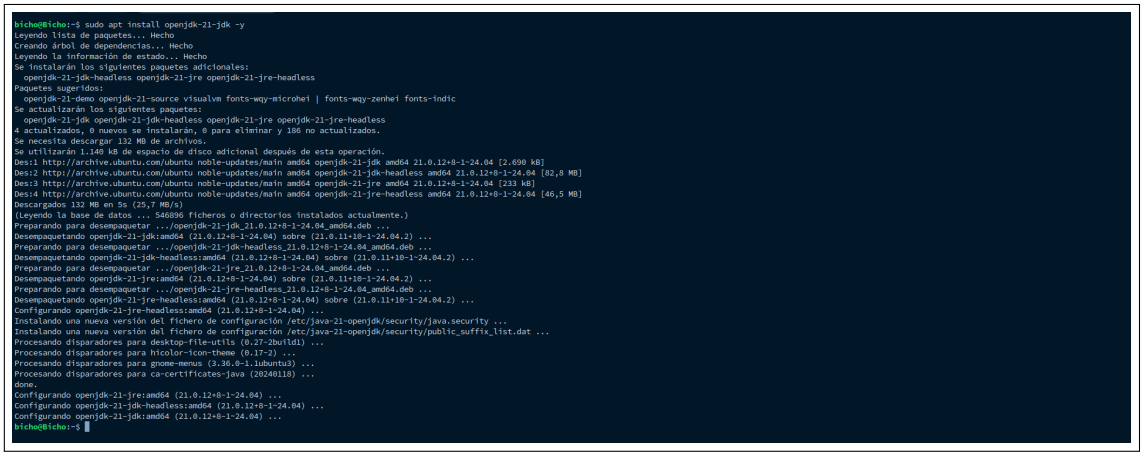


Figura 2.8: Proceso de instalación de OpenJDK 21

2.3.4. Verificación de la instalación

```
java -version      # entorno de ejecucion
javac -version     # compilador: ahora si debe responder
```

Código 2.9: Comprobación de que el JDK quedó instalado

Ambos comandos deben reportar la versión 21. Que `javac` responda es la señal de que se instaló un JDK y no solo un JRE.

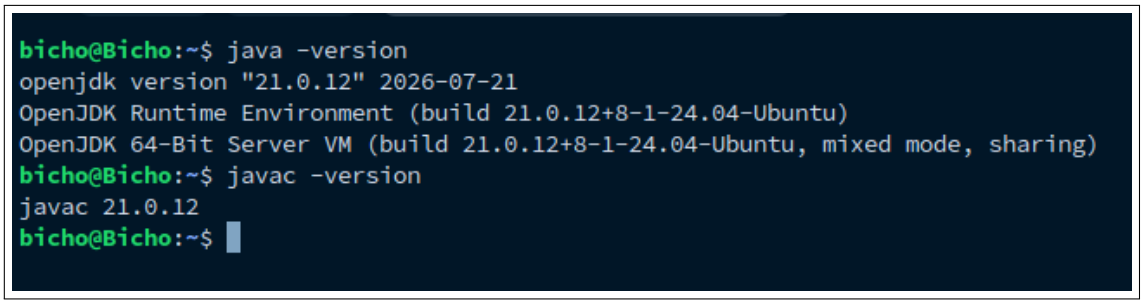


Figura 2.9: Versiones de `java` y `javac` tras la instalación

2.4. Fase 4: configuración de las variables de entorno

Esta es la parte central de la práctica y la que más errores concentra.

2.4.1. Identificación de la ruta de instalación

`JAVA_HOME` debe apuntar al **directorio raíz** del JDK: aquel que contiene las carpetas `bin`, `lib` y `conf`. No al ejecutable, ni al subdirectorio `bin`.

```
# Ruta real del compilador, resolviendo los enlaces simbolicos.
# Devuelve algo como /usr/lib/jvm/java-21-openjdk-amd64/bin/javac
readlink -f $(which javac)

# Subir dos niveles desde el binario da el directorio raiz del JDK
dirname $(dirname $(readlink -f $(which javac)))

# Contenido del directorio: deben aparecer bin, lib y conf
ls /usr/lib/jvm/
ls /usr/lib/jvm/java-21-openjdk-amd64/
```

Código 2.10: Localización del directorio raíz del JDK

En Ubuntu la ruta resultante es `/usr/lib/jvm/java-21-openjdk-amd64`.



Figura 2.10: Identificación del directorio raíz de la instalación del JDK

2.4.2. Definición de las variables

Hay que decidir el alcance de la configuración, y la elección tiene consecuencias:

Tabla 2.1: Archivos donde puede definirse `JAVA_HOME` y su alcance

Archivo	Alcance	Cuándo conviene
`~/.bashrc`	Un usuario, solo en terminales interactivas	Es el caso más común en un equipo personal. No lo leen los servicios del sistema ni, en algunas configuraciones, las aplicaciones lanzadas desde el menú gráfico.
`~/.profile`	Un usuario, también en la sesión gráfica	Cuando se necesita que los programas abiertos desde el escritorio —un IDE, por ejemplo— también vean la variable.
`/etc/environment`	Todo el sistema	Equipos con varios usuarios. No admite sustituciones ni la sintaxis `export`: solo pares `CLAVE=valor`.
`/etc/profile.d/*.sh`	Todo el sistema	La opción más limpia para el alcance global: es un archivo aparte y no se pierde al actualizar el sistema.

En este reporte se configuró a nivel de usuario, editando `~/.bashrc`:

```
nano ~/.bashrc
```

Código 2.11: Definición de `JAVA_HOME` y ampliación del `PATH`

Y agregando al final del archivo:

```
# --- Configuracion de Java ---
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

Código 2.12: Líneas añadidas a ~/.bashrc

Dos observaciones sobre estas dos líneas:

- `PATH` se **amplía**, no se reemplaza. La parte `:$PATH` al final conserva todo lo que ya había. Omitirla deja la sesión sin acceso a `ls`, `cd` ni ningún otro comando, y es uno de los errores más desconcertantes que pueden cometerse en este paso.
- El directorio del JDK se coloca **al principio** del `PATH`, de modo que si hubiera otra versión de Java en el sistema, esta tenga precedencia.

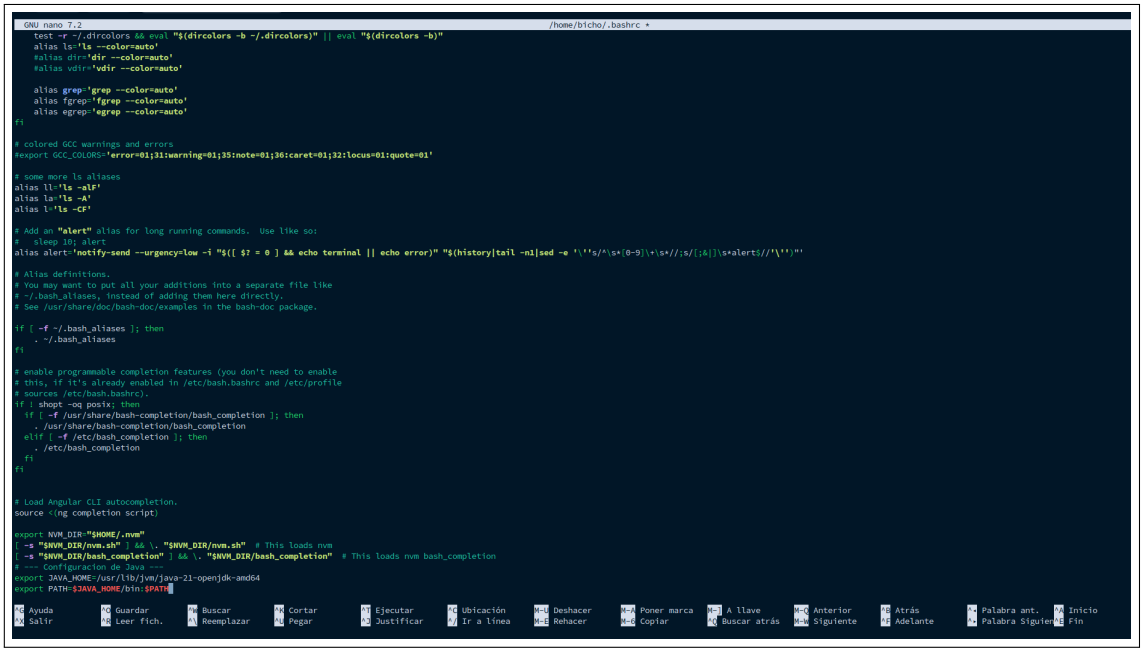


Figura 2.11: Líneas agregadas al final de `~/.bashrc`

2.4.3. Aplicación y verificación

Un archivo de configuración editado no surte efecto en la terminal que ya estaba abierta: hay que releerlo o abrir una terminal nueva.

```
# Relee el archivo en la terminal actual
source ~/.bashrc

# La variable debe mostrar la ruta del JDK
echo $JAVA_HOME

# El directorio del JDK debe aparecer al inicio del PATH
echo $PATH

# Prueba definitiva: invocar el binario a traves de la variable
$JAVA_HOME/bin/java -version
$JAVA_HOME/bin/javac -version
```

Código 2.13: Aplicación y comprobación de las variables

La última comprobación es la más significativa del reporte: si `$JAVA_HOME/bin/java` responde con la versión, entonces la variable apunta a un directorio válido. Si responde `No such file or directory`, la ruta está mal escrita o apunta a un nivel equivocado del árbol de directorios.



Figura 2.12: Variables `JAVA_HOME` y `PATH` configuradas y verificadas

2.4.4. Gestión de varias versiones instaladas

Cuando conviven varios JDK, Debian y Ubuntu resuelven cuál responde mediante el sistema de alternativas:

```
# Versiones registradas en el sistema
```

```
update-alternatives --list java

# Menu interactivo para elegir cual sera la predeterminada
sudo update-alternatives --config java
sudo update-alternatives --config javac
```

Código 2.14: Selección de la versión activa

Conviene entender que `update-alternatives` y `JAVA_HOME` son mecanismos **independientes**: el primero decide qué binario responde al escribir `java` en la terminal; el segundo, qué JDK usan Maven, Gradle o el IDE. Pueden apuntar a versiones distintas sin que el sistema se queje, y esa discrepancia silenciosa explica buena parte de los problemas de compilación difíciles de diagnosticar.



Figura 2.13: Versiones registradas en el sistema de alternativas

2.5. Fase 5: verificación funcional

Que los comandos reporten una versión no demuestra que el kit funcione. La prueba real es compilar y ejecutar un programa.

```
mkdir -p ~/pruebaJava && cd ~/pruebaJava
nano HolaMundo.java
```

Código 2.15: Creación del programa de prueba

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("El JDK quedo instalado correctamente.");
        System.out.println("Version de Java : " + System.getProperty("java.version"));
        System.out.println("Proveedor       : " + System.getProperty("java.vendor"));
        System.out.println("Directorio home : " + System.getProperty("java.home"));
    }
}
```

Código 2.16: Programa de prueba (`HolaMundo.java`)

El programa no se limita a imprimir un saludo: consulta las propiedades del sistema para que la salida muestre qué JDK está ejecutándolo realmente. El valor de `java.home` debe coincidir con el de `JAVA_HOME`; si no coincide, algo de la configuración quedó inconsistente.

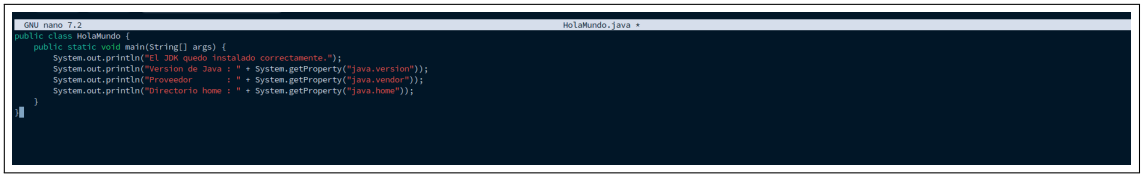


Figura 2.14: Código fuente del programa de prueba

```
# Compilar: genera HolaMundo.class
javac HolaMundo.java

# Verificar que se creo el bytecode
ls -l

# Ejecutar (sin la extension .class)
java HolaMundo
```

Código 2.17: Compilación y ejecución

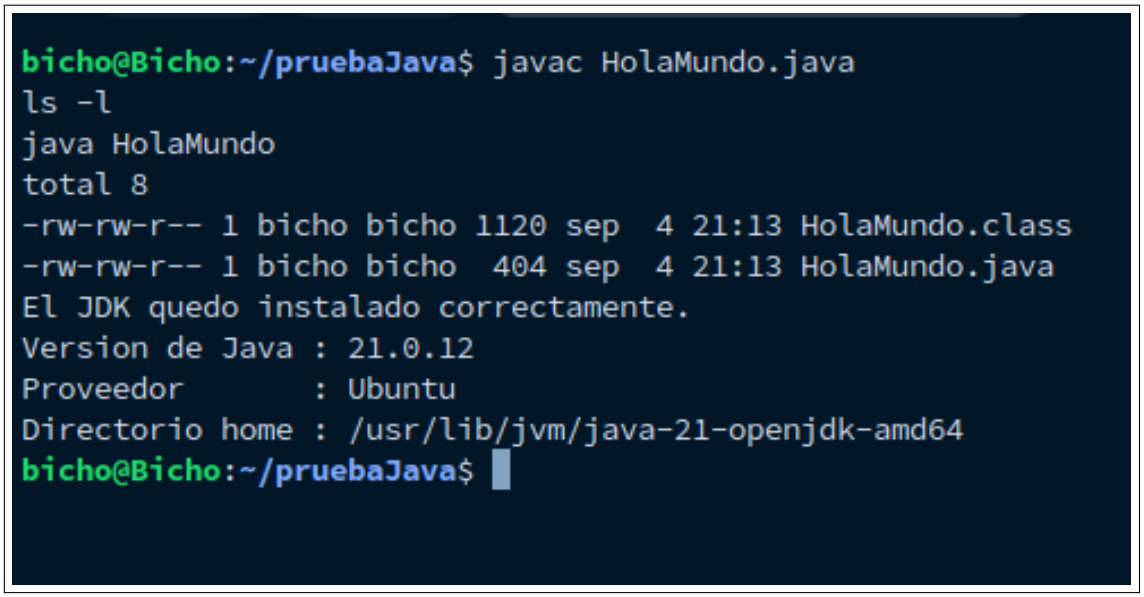


Figura 2.15: Compilación y ejecución del programa de prueba

Con esta última evidencia queda demostrado el ciclo completo: el código fuente se compiló a *bytecode* con `javac` y la JVM lo ejecutó, usando el JDK señalado por `JAVA_HOME`.

3. Conclusiones

3.1. Sobre el proceso

Primera: instalar el JDK y configurarlo son dos tareas distintas, y la segunda es la que realmente se estaba practicando. La instalación se resuelve con un solo comando de APT y difícilmente falla. Lo que sí falla —y lo que ninguna herramienta avisa— es la configuración de las variables de entorno: un sistema puede tener el JDK perfectamente instalado, responder correctamente a `java -version` y aun así ser incapaz de compilar un proyecto con Maven, porque `JAVA_HOME` no existe o apunta a otro lado.

Segunda: la distinción entre JRE y JDK explica el error más frecuente. El estado inicial del equipo documentado en este reporte —una JVM funcionando sin compilador— es exactamente el que produce la confusión de creer que "Java ya está instalado". La comprobación que separa un caso del otro no es `java -version` sino `javac -version`.

Tercera: `JAVA_HOME` **debe apuntar a un directorio, no a un ejecutable.** Es el error conceptual que más se repite. La variable nombra el **directorio raíz** de la instalación —el que contiene `bin`, `lib` y `conf`—, porque las herramientas que la consultan construyen a partir de ella rutas como `$JAVA_HOME/bin/javac` o `$JAVA_HOME/lib/tools.jar`. Apuntarla al binario rompe todas esas construcciones a la vez.

Cuarta: la persistencia depende del archivo elegido, no del comando. Escribir `export JAVA_HOME=...` directamente en la terminal funciona hasta que se cierra la ventana. Que la configuración sobreviva depende de haberla escrito en el archivo correcto, y ese archivo cambia según quién necesite ver la variable: el usuario en su terminal, las aplicaciones gráficas, o los servicios del sistema.

3.2. Errores posibles durante el proceso y su solución

El siguiente catálogo reúne los problemas que pueden presentarse, ordenados por frecuencia, con su causa y la forma de corregirlos.

Tabla 3.1: Errores frecuentes en la instalación y configuración del JDK

Síntoma	Causa	Solución
`javac: command not found` pero `java` sí funciona	Se instaló el paquete `jre` en lugar del `jdk`	`sudo apt install openjdk-21-jdk`. El sufijo del paquete es lo que distingue uno de otro.
`echo $JAVA_HOME` sigue vacío tras editar el archivo	No se releyó la configuración: la terminal abierta conserva el entorno anterior	`source ~/.bashrc`, o cerrar y abrir la terminal.
`$JAVA_HOME/bin/java: No such file or directory`	La variable apunta al ejecutable o al subdirectorio `bin` en vez de a la raíz del JDK	Usar `/usr/lib/jvm/java-21-openjdk-amd64`, sin `/bin` ni `/bin/java` al final.
Maven o Gradle reportan `JAVA_HOME is not set` aunque `java` funcione	La variable se definió solo para terminales interactivas y el proceso no la hereda	Definirla en `~/.profile` o en `/etc/profile.d/`, y cerrar la sesión para que surta efecto.
La configuración se pierde al cerrar la terminal	Se ejecutó `export` en la línea de comandos, sin escribirlo en ningún archivo	Agregar las líneas a `~/.bashrc` y volver a cargarlo.
Tras editar el `PATH`, ningún comando funciona (`ls`, `cd`...)	Se sobrescribió el `PATH` en vez de ampliarlo	Recuperar la sesión con `export PATH=/usr/bin:/bin` y corregir la línea para que termine en `:$PATH`.
`java -version` reporta una versión distinta a la instalada	Hay varias versiones y el sistema de alternativas apunta a otra	`sudo update-alternatives -config java` y elegir la deseada.
El IDE compila con una versión y la terminal con otra	`JAVA_HOME` y `update-alternatives` apuntan a JDK distintos	Alinear ambos, y revisar además el JDK configurado dentro del propio IDE.
`E: se interrumpió la ejecución de dpkg` en cualquier comando de APT	Una instalación anterior de **cualquier** paquete quedó a medio configurar, y APT se bloquea hasta repararlo	`sudo dpkg -configure -a`, después `sudo apt -fix-broken install`. Diagnosticar con `sudo dpkg -audit`.
`Unable to locate package openjdk-21-jdk`	Índices de APT desactualizados, o la versión no existe para esa distribución	`sudo apt update` y verificar con `apt-cache policy`. En versiones antiguas de Ubuntu puede hacer falta un PPA.
`404 Not Found` a mitad de la descarga	El catálogo local apunta a una versión que el repositorio ya reemplazó	`sudo apt update` y repetir la instalación.
`Permission denied` al editar `/etc/environment`	El archivo es del sistema y requiere privilegios	Usar `sudo nano /etc/environment`.
`/etc/environment` no surte efecto	Se escribió con la sintaxis `export CLAVE=valor`	Ese archivo solo admite pares `CLAVE="valor"`, sin `export`.
`apt autoremove` propone eliminar programas en uso	Aplicaciones de escritorio que dependen del Java que se está retirando	Revisar la lista antes de aceptar; si hay dudas, no desinstalar y convivir con ambas versiones.
Todo funciona hasta reiniciar, y entonces se pierde	La variable se definió en un archivo que la sesión nueva no lee	Verificar el archivo elegido según la tabla 2.1 y comprobar con una terminal recién abierta.
`UnsupportedClassVersionError` al ejecutar	El *bytecode* se compiló con una versión más nueva que la JVM que lo ejecuta	Alinear las versiones, o compilar con `javac -release 21`.

3.3. Recomendaciones

1. **Diagnosticar antes de instalar.** Ejecutar `javac -version` y `echo $JAVA_HOME` antes de tocar nada evita instalar sobre una configuración que ya existía y que quedará huérfana.
2. **Verificar con** `$JAVA_HOME/bin/java -version`, no solo con `echo $JAVA_HOME`. Lo primero

comprueba que la ruta **existe y es válida**; lo segundo solo muestra una cadena de texto que puede estar mal escrita.

3. **Comprobar en una terminal nueva.** Es la única forma de saber si la configuración es persistente o si solo vive en la sesión actual.
4. **Usar el enlace simbólico y no la ruta con número de versión.** En Ubuntu, `/usr/lib/jvm/java-21-openjdk-amd64` se mantiene estable entre actualizaciones menores, mientras que una ruta como `.../jdk-21.0.11` deja de existir en cuanto el paquete se actualiza a `21.0.12`, y `JAVA_HOME` queda apuntando a la nada.
5. **Para varias versiones, considerar SDKMAN.** En equipos de desarrollo que alternan entre proyectos con distintas versiones de Java, una herramienta de gestión como SDKMAN evita reconfigurar variables a mano. Conviene saber que ese tipo de herramientas también modifica `JAVA_HOME`, y que mezclarla con una configuración manual en `.bashrc` produce conflictos difíciles de rastrear.

3.4. Sobre la distribución utilizada

Se documentó la instalación de OpenJDK en lugar de Oracle JDK. Vale la pena cerrar señalando qué habría cambiado y qué no.

No habría cambiado nada de lo esencial. Ambas distribuciones parten del mismo código fuente del proyecto OpenJDK, superan el mismo conjunto de pruebas de compatibilidad y ejecutan idénticamente el mismo *bytecode*. Todo lo relativo a `JAVA_HOME`, al `PATH`, al sistema de alternativas y a los errores catalogados arriba aplica exactamente igual.

Habría cambiado el procedimiento de obtención y la ruta. Con Oracle JDK habría que descargar el paquete `.deb` desde el sitio de Oracle aceptando su licencia, instalarlo con `sudo dpkg -i`, y `JAVA_HOME` apuntaría a `/usr/lib/jvm/jdk-21-oracle`. A cambio de ese trabajo manual se pierde la actualización automática por APT: cada parche de seguridad exigiría repetir la descarga.

La diferencia real es de licencia. OpenJDK se distribuye bajo GPLv2 con excepción de enlazado, sin restricciones de uso. Oracle JDK se rige por los *Oracle No-Fee Terms and Conditions*, que permiten el uso gratuito bajo condiciones específicas y han cambiado varias veces en los últimos años. Para un entorno académico y para la mayoría de los proyectos, OpenJDK es la opción con menos fricción legal y operativa.

Referencias

Canonical Ltd. (2026). *Ubuntu manpage: update-alternatives — maintain symbolic links determining default commands*. https://manpages.ubuntu.com/manpages/jammy/man1/update-alternatives.1.html

Canonical Ltd. (2026). *Ubuntu packages: openjdk-21-jdk (jammy-updates)*. https://packages.ubuntu.com/jammy-updates/openjdk-21-jdk

Free Software Foundation. (2026). *Bash reference manual: Bash startup files*. https://www.gnu.org/software/bash/manual/html_node/Bash-Startup-Files.html

Oracle Corporation. (2026). *Installation of the JDK on Linux platforms*. JDK 21 Installation Guide. https://docs.oracle.com/en/java/javase/21/install/installation-jdk-linux-platforms.html

Oracle Corporation. (2026). *Java Platform, Standard Edition 21 tools reference: javac*. https://docs.oracle.com/en/java/javase/21/docs/specs/man/javac.html

Oracle Corporation. (2026). *Java SE support roadmap*. https://www.oracle.com/java/technologies/java-se-support-roadmap.html

Oracle Corporation. (2026). *Oracle No-Fee Terms and Conditions (NFTC)*. https://www.oracle.com/downloads/licenses/no-fee-license.html

OpenJDK Community. (2026). *JDK 21*. https://openjdk.org/projects/jdk/21/

OpenJDK Community. (2026). *The GNU General Public License, version 2, with the Classpath Exception*. https://openjdk.org/legal/gplv2+ce.html

The Apache Software Foundation. (2026). *Maven installation: configuring JAVA_HOME*. https://maven.apache.org/install.html